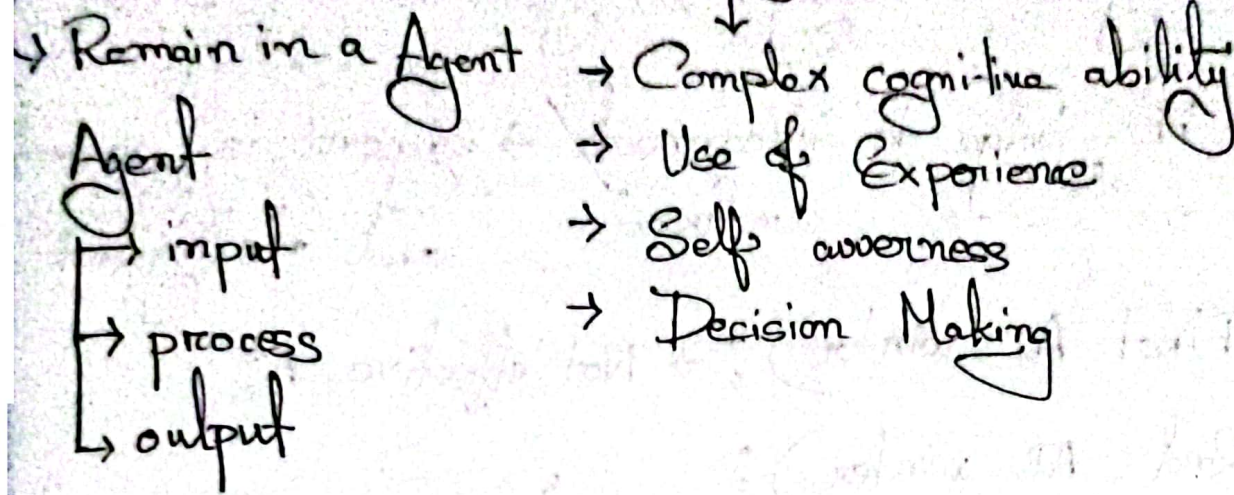


(Lecture : 1)

CSE 422

AI ? Artificial Intelligence (term invented in 1950)
(get recognized in 1956)



AI : (Pillars)

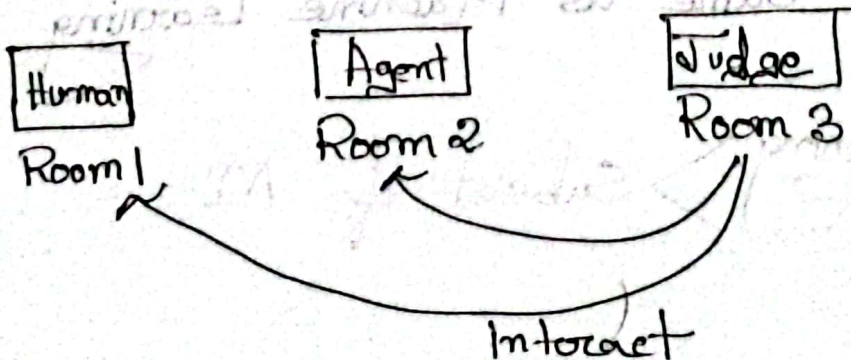
→ Store knowledge (Experience)

→ Reason about knowledge (Accomplish a task using knowledge)

→ Behave intelligently (Any complex decision making)

→ Interact with people, agents and environment

Turing Test :



Judge interact with human and agent

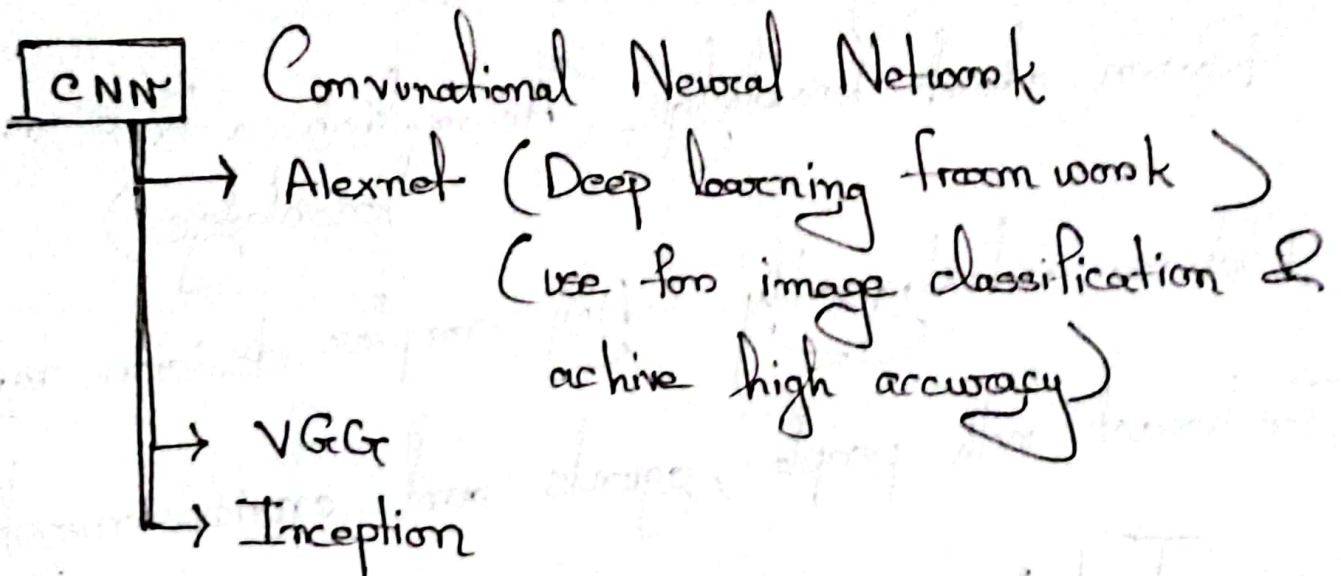
→ if agent can give or can detect who's agent correctly → agent is not smart

→ if judge is incorrect / confused → agent is smart and intelligent

1970 (First AI winter) : → Not effective AI

1980 (2nd AI winter) :

After 2000 progress of AI start



AI is not same as Machine Learning

Machine learning > Subset of AI
Deep " >

Deep Learning → has no manual implementation
Dynamical learning

Due to AI Progress

→ loss of job

→ AI rationality (Try to achieve goal in most efficient way but it could be wrong/unethical)

→ Stop button

- (may not work)

Behavior:

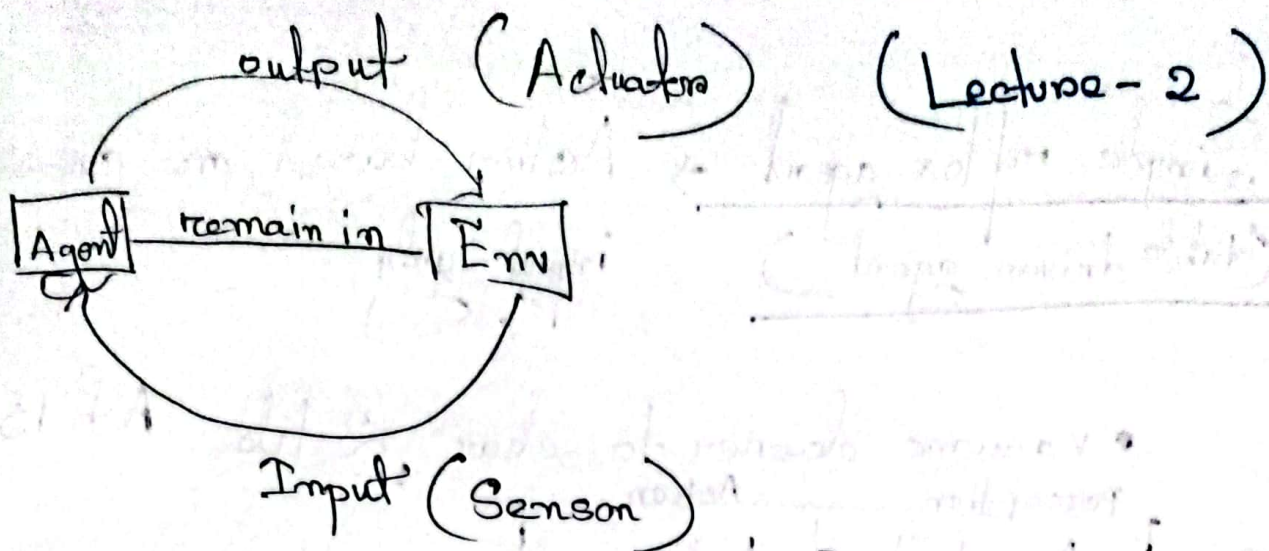
↳ Human like behavior (think and use emotion)

↳ Rational " (just do things to achieve highest profit will not consider any other ethical or emotional issue)

Lecture - 2

How to design an intelligent agent?

- An intelligent agent -
 - perceive environment via sensors
 - act rationally with its actuators
- A discrete agent -
 - receive percepts one at a time
 - maps this percept sequence to a sequence of discrete actions.
- Properties
 - Autonomous.
 - Reactive to the environment
 - Pro-active
 - Interact with other agents



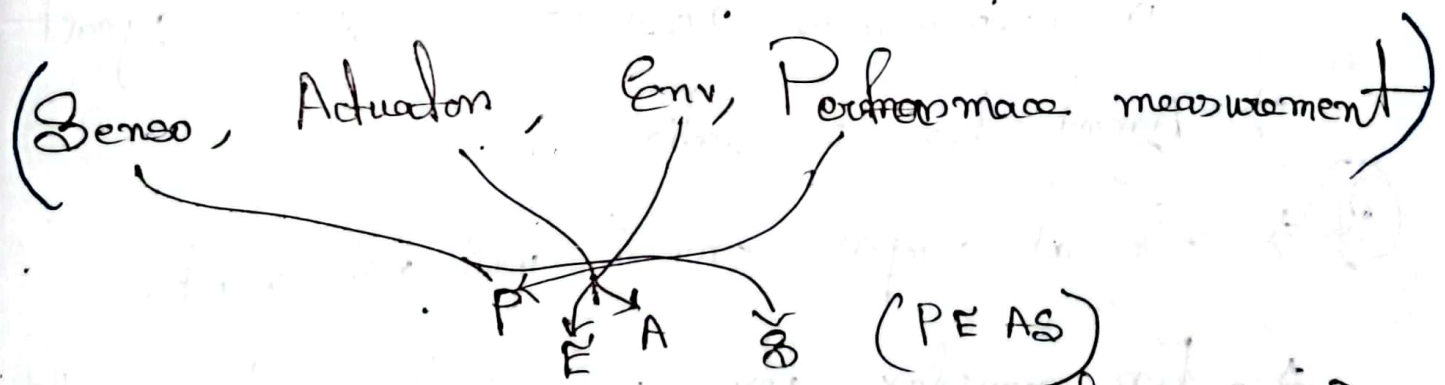
Input can be $\rightarrow$ eyes, ears, etc

Input take through module $\rightarrow$ sensors

Output generate in " $\rightarrow$ Actuators

Same element may work as Actuators & Sensors

positive impact $\rightarrow$ performance high else low



Agent $\begin{cases} \text{simple (light on off on basis of day light)} \\ \text{complex (Self driving car)} \end{cases} \rightarrow \text{module complex pipeline "}$

processing function "

i) Simple reflex agent → Action based on current input only.
(table driven agent)

• Vacuum cleaner to clean 2-lobes A & B

⇒ Use action table to find the next action

Perception		Action
A	dusty	A clean
A	clean	Go B
B	dusty	Clean B
B	clean	Go A

① ⇒ Cannot deal with continuously changing environment (No scalable)

② ⇒ Agent is not dynamic for every change in environment need to change the agent

⇒ Agent here is static

③ ⇒ So at complex env it will not work

⇒ For complex env, perception action table can go to infinity no of action

④ ⇒ Prone to looping

Disadvantage

- ⇒ Not scalable
- ⇒ Not suitable for long env
- ⇒ loop

ii) Agent with memory / Model based agent :

- A clean → set to memory its clean
- Memory is dynamic
- Have internal state that is used to keep track of past state.

iii) Goal Based Agent : (Rational behaviors) :

Act on the basis of goal rather than
performing action on the ~~goal~~ based
table → perception

- have goal information that describe desirable situations.

Input → Configuration of perception and action

goal → Number of dirty tile 0

no of dirty tile = 2

heuristic

↓
A particular agent is at what distant from its targeted goal

iv) Utility Based Agent :

Multimodal heuristic

Utility function $\rightarrow$ side goal (may be for safety)

To act goal $\rightarrow$ has to achieve (Primary)

utility $\rightarrow$ not necessary to achieve



Happiness Score $\rightarrow$ from utility function

Performance evaluation matrix $\rightarrow$ low

- take decision on classic axiomatic utility theory

Properties of Environment :

i) Partially observable vs fully observable
(fully observe করে কিনা sensor)

ii) Deterministic vs. stochastic:

Deterministic → for one action the output is always correctly predicted / known.

Stochastic → environment not/cannot be predicted on the basis of action.

iii) Episodic vs sequential:

↳ not linked (action is not linked with others)
↳ linked

iv) Static vs dynamic:

↳ no change in environment
↳ change in environment
(Real world ")

v) Single agent vs Multi agent:

↳ opponent $\pi/2$
↳ involvement of opponent

§ vi) Discrete vs continuous:

- environment defined ~~not~~ using finite no.
- cannot defined using finite numbers

vii) Known vs Unknown:

- after learning from data

(Lecture-3)

Informed Search (More Efficient)

- Greedy Best First Search
- A* Search

8 Puzzle game

Goal State equal -

1	2	3
4	5	6
7	8	

Here we will use heuristic value -
ie is how much far it is from goal state

Let consider -

$h = 5$

1	2	3
5	↑ ↓	8
4	6	7

→ Start State

→ heuristic (h) = No of misplaced tile

$h=6$

1		3
5	2	8
4	6	7

$h=5$

1	2	3
5	6	8
4		7

$h=4$

1	2	3
	5	8
4	6	7

$h=4$

1	2	3
5	8	
4	6	7

$h=5$

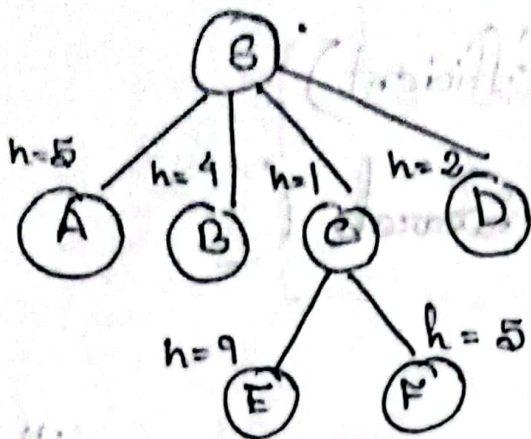
	2	3
1	5	8
4	6	7

$h=3$

1	2	3
4	5	8
	6	7

So we will chose
whose h value
is less - means
closer to goal

Let a Graph be: (8-nodes)



B	5
A	5
B	4
C	1
D	2
E	9
F	5
I	0

→ Goal State

8 puzzle:

$\hookrightarrow h_1 =$ No of misplaced tile
 $\hookrightarrow h_2 =$ Manhattan distance (better)

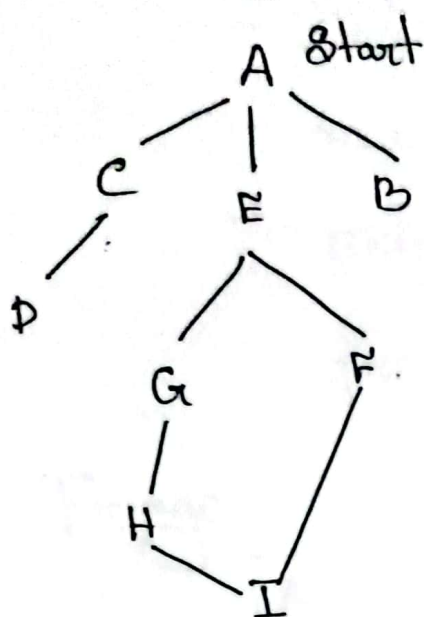
h = at how much distance the misplaced tile is.

1	3	2
4	7	8
6	5	

$$\begin{array}{ccccccc}
 3 & 2 & 7 & 8 & 6 & 5 \\
 \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\
 h = 1 + 1 + 2 + 2 + 3 + 1 \\
 = 10
 \end{array}$$

- Manhattan distance value is always large & longer the better. So h_2 is better.
- Here h_2 dominate h_1
- ie if $h_1 \geq h_2$; h_1 dominates h_2

i) Greedy Best First Search :



A	366
B	374
C	329
D	244
E	256
F	178
G	193
H	98
I	0

Fringe : (Priority Queue)

A³⁶⁶

A³⁶⁶ C³²⁹ E²⁵³ B³⁷⁴

(here lowest edge value will be pop)

lowest h value ← E²⁵³

C³²⁹ B³⁷⁴ G¹⁹³ F¹⁷⁸

F¹⁷⁸

C³²⁹ B³⁷⁴ G¹⁹³ I⁰

I⁰

C³²⁹ B³⁷⁴ G¹⁹³

(As I⁰ expand & pop so the fringe terminate)

Here path is - A → E → F → I

path cost = 450 (Not Optimal)

As we get less path cost in different way

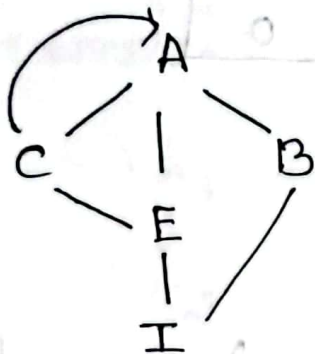
GBFS $\rightarrow$ Not Optimal

$\rightarrow$ Complete?

Search has 2 versions

- Tree version
- Graph version

Tree Version:



A	1
B	3
C	2
E	4
I	0

Fringe:

A'

A' [C² E⁴ B³]

C [A² E⁴ B³]

A [E⁴ B³ C² E⁴]

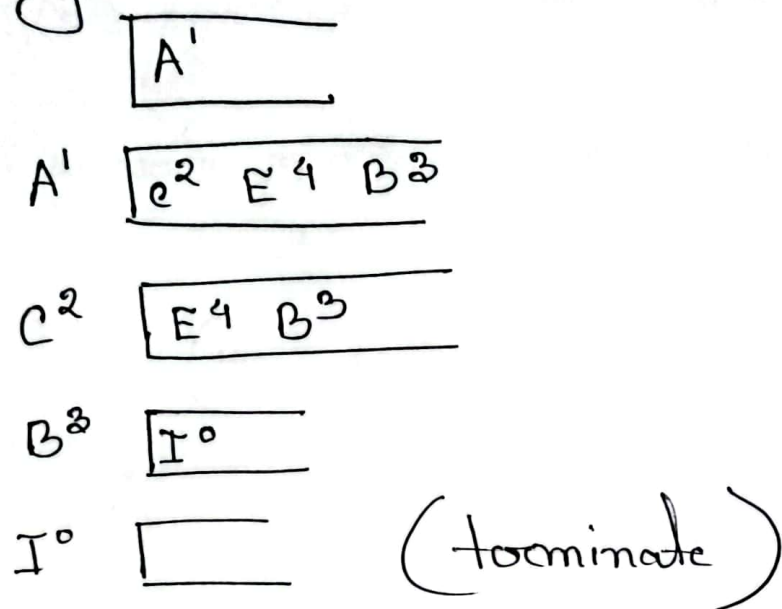
create loop
C and A repeat
again and again

- So tree GBFS is not complete (create loop)

Graph Search :

- Used to prevent loop
- mark visited node
- skip the visited node while expanding/
pushing into the fringe.

Fringe



Q. Complete ?

- Tree version → Incomplete
- Graph version → finite → complete

Infinite → incomplete

Time & Space Complexity - $O(b^m)$

b - branching factor
m - max depth

(Lecture - 4)

Informed Search (Single Player Game)

(A* Search)

$$f(n) = g(n) + h(n)$$

$h(n)$ = heuristic value

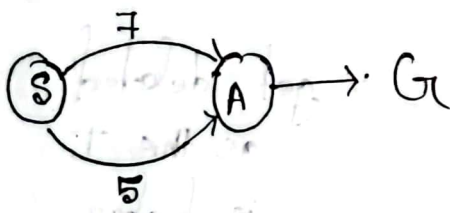
$g(n)$ = Actual path cost from start node (lowest)

For node $n \rightarrow F$ (From slide)

$$g(n) + h(n)$$

$$(140 + 99) \downarrow 178$$

$$\therefore f(n) = 417$$



$$f(A) = g(A) + h(A)$$
$$\downarrow \quad + \quad ()$$
$$5 \quad + \quad ()$$

$$f(D) = g(D) + h(D) \rightarrow (\text{from slide}) \rightarrow 11 \rightarrow 2$$
$$= 299 + 244$$
$$= 473$$

Graph version of A^*

Fringe:

$$A \quad h(A) + g(A) = 366 + 0 = 366$$

$$A^{366} \quad \begin{array}{l} B \quad 374 + 78 = 449 \\ C \quad 329 + 118 = 447 \\ E \quad 253 + 140 = 393 \end{array}$$

$$E^{393} \quad \begin{array}{l} B^{449} \quad C^{447} \quad G \quad 193 + (140 + 80) = 413 \\ F \quad 178 + (239) = 417 \end{array}$$

$$G^{413} \quad \begin{array}{l} B^{449} \quad C^{447} \quad \cancel{D^{448}} \quad F^{417} \quad H \quad 98 + (140 + 80 + 97) = 415 \end{array}$$

$$H^{415} \quad \begin{array}{l} B^{449} \quad C^{447} \quad F^{417} \quad I \quad 0 + 418 = 418 \end{array}$$

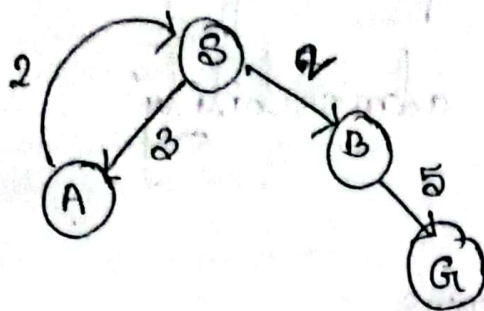
$$F^{417} \quad \begin{array}{l} B^{449} \quad C^{447} \quad I^{418} \quad \cancel{J} \quad 0 + 450 = 450 \end{array}$$

$$I^{418} \quad \begin{array}{l} B^{449} \quad C^{447} \end{array}$$

get deleted
as this I
is higher

$$I \leftarrow H \leftarrow G \leftarrow E \leftarrow A$$

$\therefore A^*$ search is Complete



S	2
B	7
A	3
G	0

GBFS:

Fringe:

S^2	
S^2	$A^3 \quad B^7$
A^3	$B^7 \quad S^2$
S^2	$B^7 \quad A^3$
A^3	$B^7 \quad S^2$

Fall in loop so will never reach goal

A* Search:

Fringe:

	S^{2+0}
S^2	$A^{3+3=6} \quad B^{7+2=9}$
A^6	$B^9 \quad S^{2+(3+2)=7}$
S^7	$B^9 \quad A^{3+8=11}$
B^9	$A^{11} \quad G^{0+7=7}$
G^7	A^{11}

$S^2 \rightarrow A^3 \rightarrow S^2 \rightarrow A^6$

$\therefore$ (A* search is optimal)

☐ A^* Search becomes optimal if it follow the properties of h -admissibility

h -admissibility means

if heuristic cost can ^{be} underestimated

or less by / from actual cost then

that graph is h -admissible.

Actual cost = F to T path cost = 211

Heuristic cost of F

= 178

$\therefore 178 < 211$

So when for every node the heuristic value is less or is underestimated by actual cost then A^* search is optimal & admissible.

(Limiting of heuristic h)

Fringe : (After change value from table { H-140 })

$$A^{866} + 0 = 866$$

A^{866}

$$B^{449} \quad C^{447} \quad E^{893}$$

E^{893}

$$B^{449} \quad C^{447} \quad G^{413} \quad F^{417}$$

G^{413}

$$B^{449} \quad C^{447} \quad F^{417} \quad H^{457}$$

F^{417}

$$B^{449} \quad C^{447} \quad H^{457} \quad I^{450}$$

C^{447}

$$B^{449} \quad H^{457} \quad I^{450} \quad D^{473}$$

B^{449}

$$H^{457} \quad I^{450} \quad D^{473}$$

I^{450}

$$H^{457} \quad D^{473}$$

$A \rightarrow E \rightarrow F \rightarrow I$ (Not optimal)

As the heuristic here not
admissible

H-Consistency :



S	5
A	2
G	

it can be.
($n \leq 6$)

$$h(A) \leq c(A, G) + h(G)$$

$$2 + 0 = 2$$

$$h(S) \leq c(S, A) + h(A)$$

$$4 + 2 = 6$$

Consistent condition

that is the $h(n)$ of a particular node need to be less or equal from its path cost of success value and its successor's heuristic value

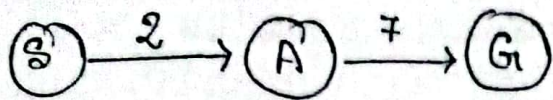
$$\text{ie } h(n) \leq c(n, n+1) + h(n+1)$$



$$\text{Now } h(S) = c(S, A) + h(A)$$

$$= 3 + 2$$

5



A	2
S	6
G	0

The graph is admissible

$$h(A) \leq c(A, G) + h(G)$$

$$7 + 0 = 7$$

$$h(S) \leq c(S, A) + h(A)$$

$$\left[\begin{array}{l} 2 + 2 = 4 \text{ (condition not fulfilled)} \\ \rightarrow \text{so not consistent} \end{array} \right.$$

If a graph is consistent it must be on
it is always admissible.

If the graph is not consistent $\rightarrow$ It may have
many irrelevant branching to reach
the goal.

Heuristic Dominance :

$$h_2 \geq h_1$$

for each and every node h_2 returns large value than h_1 $\therefore h_2$ dominates h_1

- large value should be picked

Quiz-1

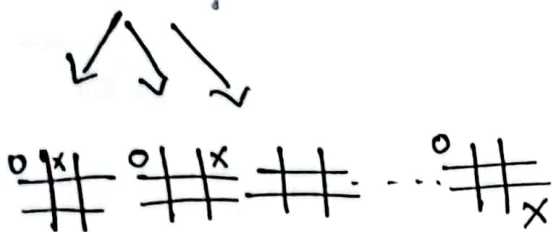
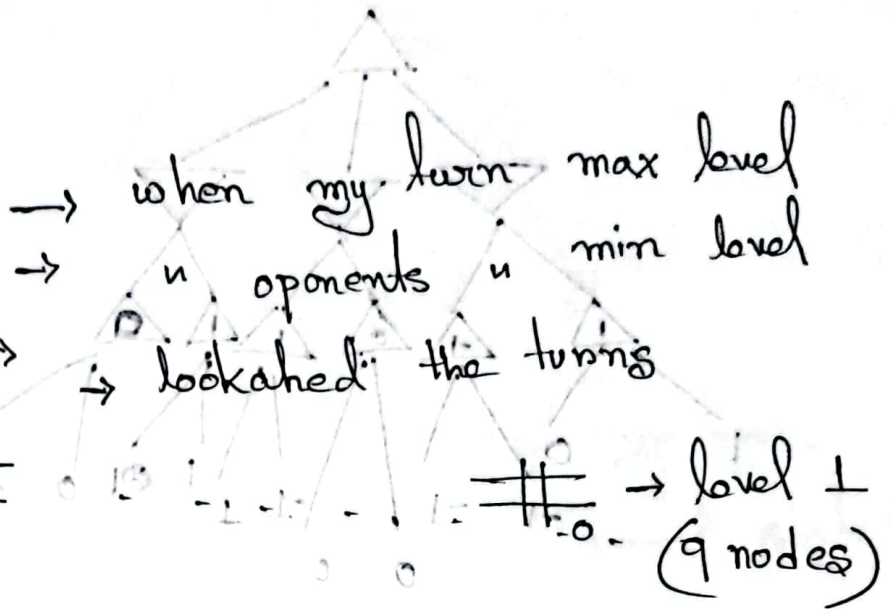
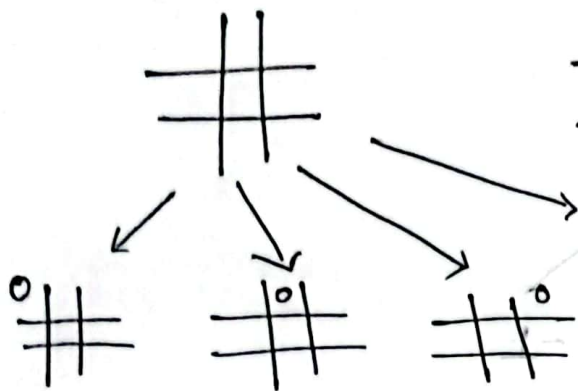
Informed Search

(Saturday)

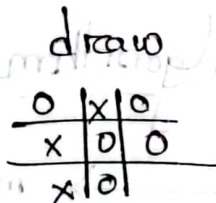
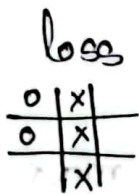
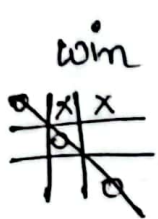
(Lecture-5)

Games $\rightarrow$ 2 player games

Tic-tac-toe



$\rightarrow$ level 2 (9×8) nodes
= 72 nodes



zero-sum game

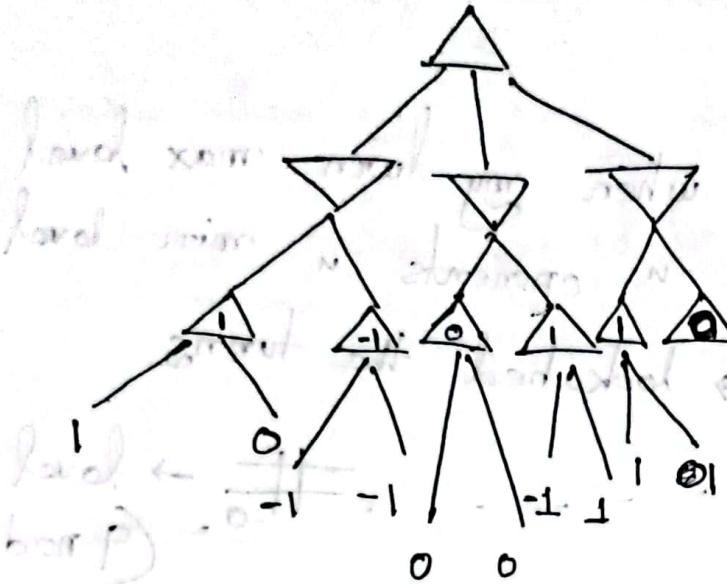
Adding gain and loss will be equal 0

If one's winning probability is maximized
other's " " " minimized

State Space (From my perspective):

max level $\rightarrow \triangle$

min level $\rightarrow \nabla$

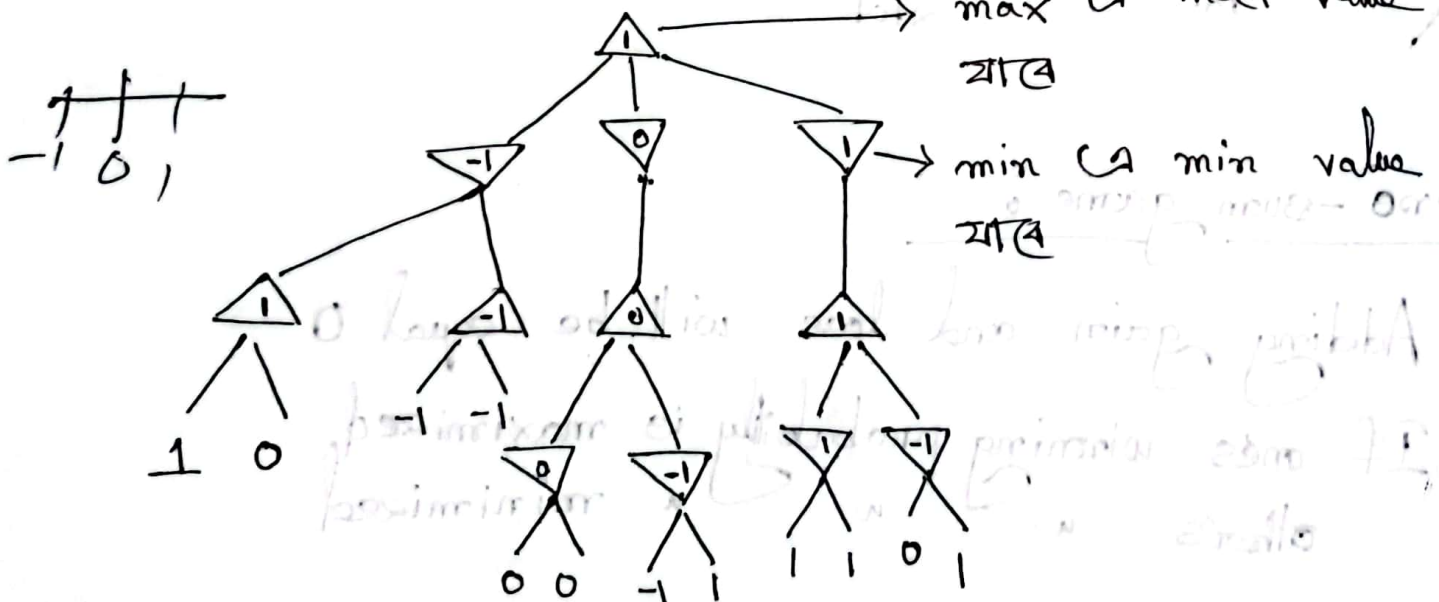


1 $\rightarrow$ win

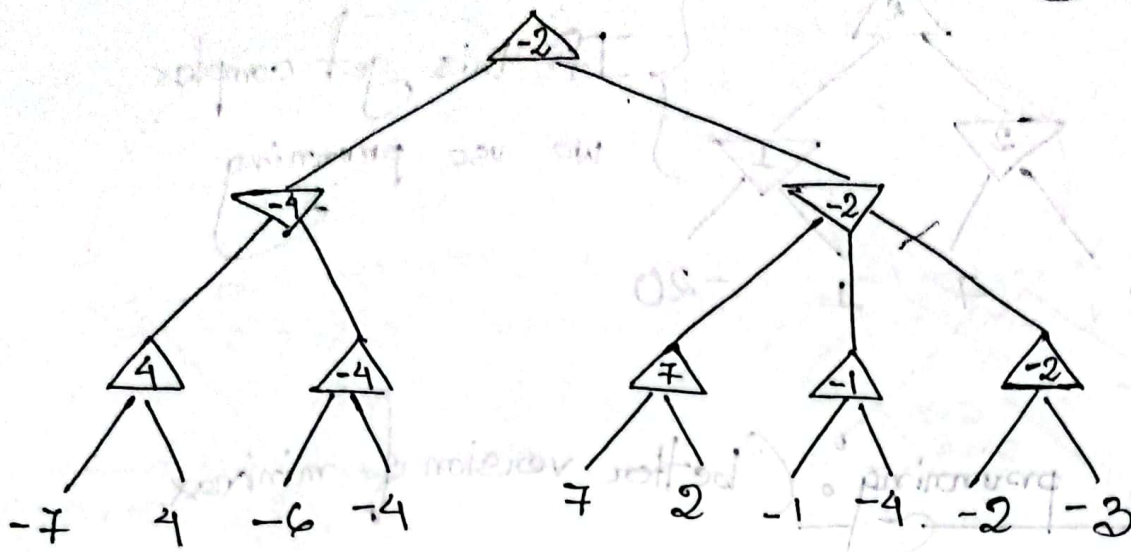
-1 $\rightarrow$ loss

0 $\rightarrow$ draw

Minimax Algorithm

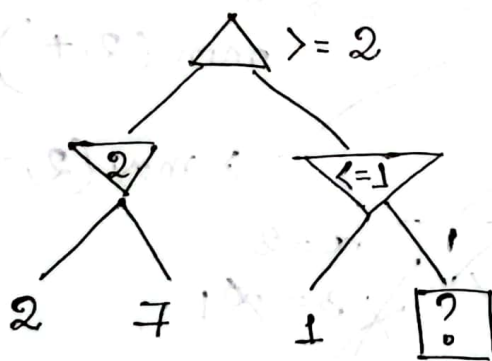


Returned value of a good function by minimax Algorithm

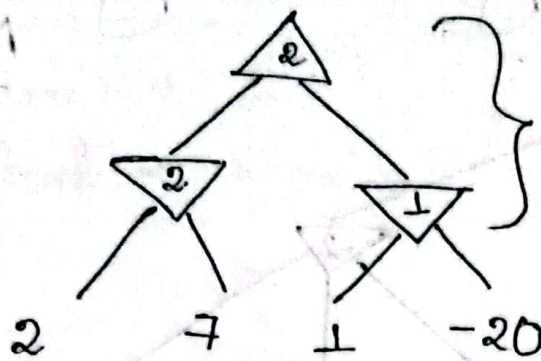


good function $\rightarrow$ mainly return positive / negative response evaluating the board state / current board state.

Intuition of value :



(Analysing the overall state it can be said that this value (?) will not effect the overall state)



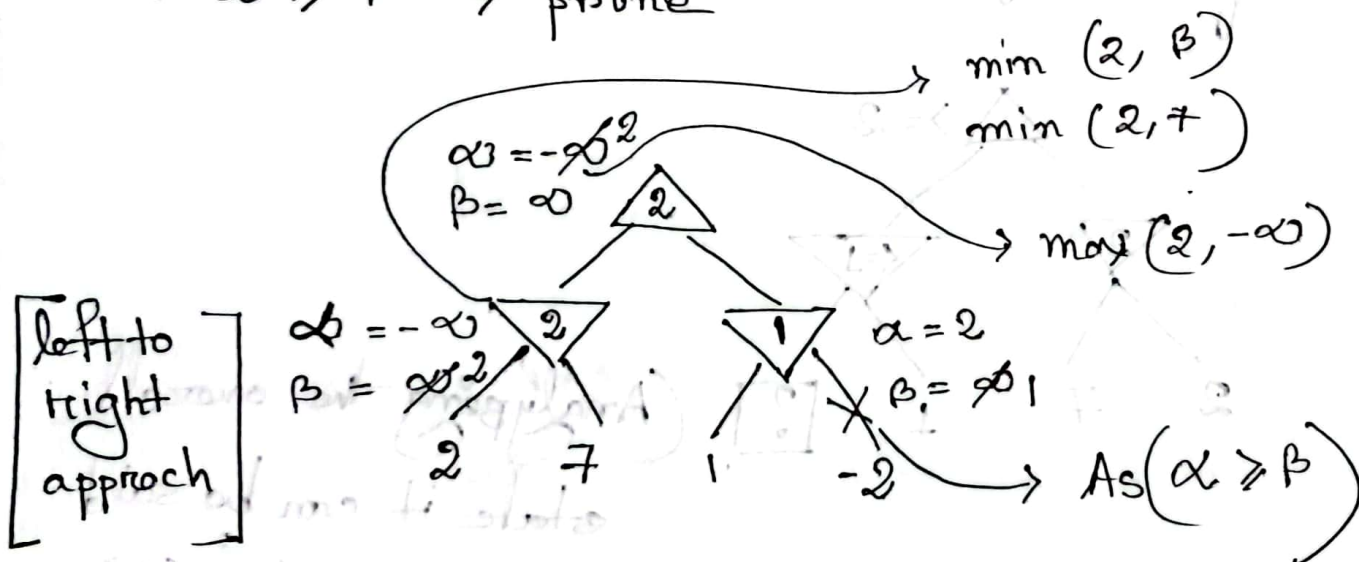
If this get complex
we use pruning

Alpha-beta pruning (better version of minimax)

$$\alpha \rightarrow -\infty$$

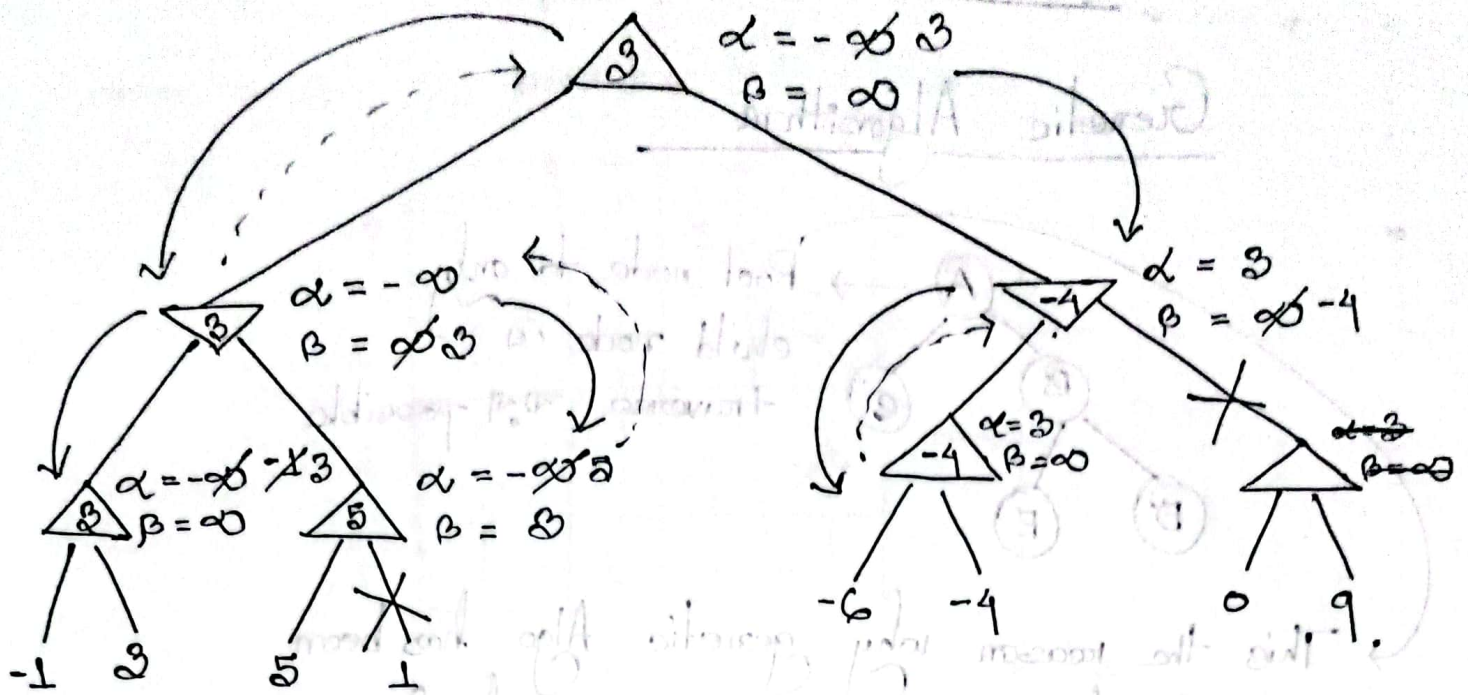
$$\beta \rightarrow \infty$$

- For max node $\rightarrow$ update $\alpha \rightarrow$ max value
- " min " $\rightarrow$ " $\beta \rightarrow$ min value
- When,
- $\alpha \geq \beta \rightarrow$ prune



If the state tree is large the extra
branch get cut out so become less
expensive and reduce time

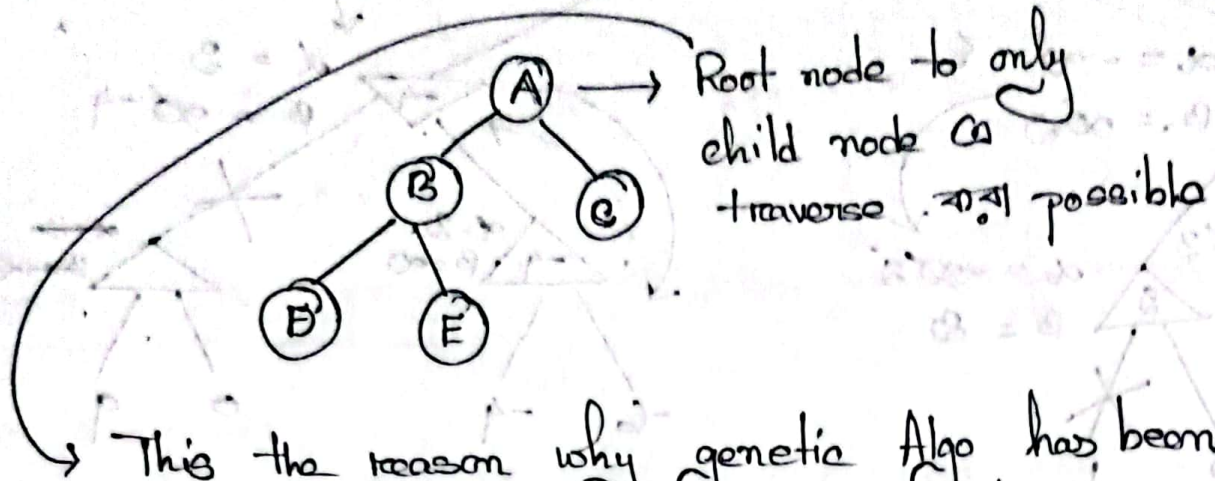
Example :



Visit $\rightarrow$ Alpha-beta pruning Stimulation (2nd page)

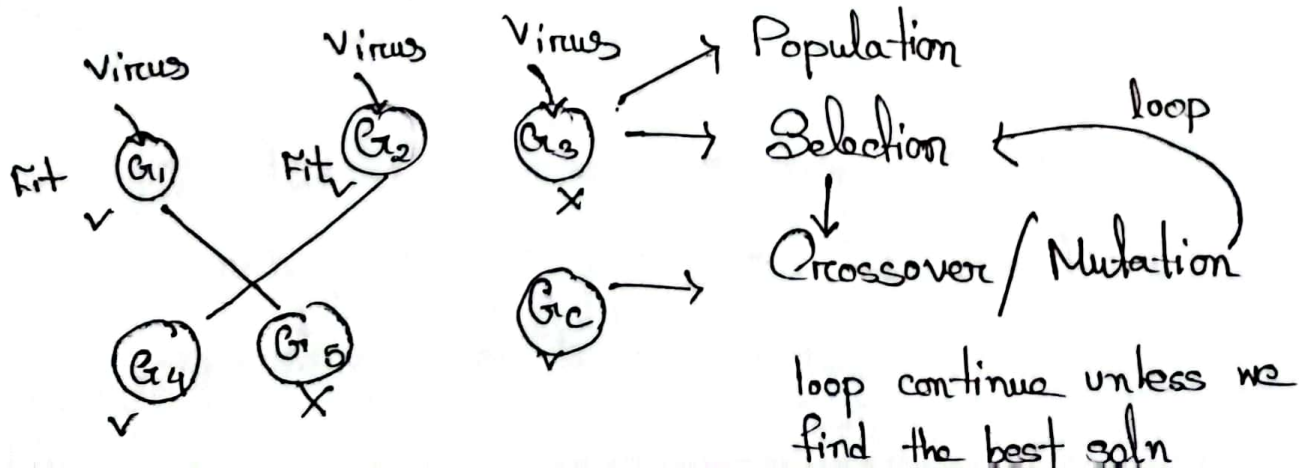
(Lecture - 6)

Genetic Algorithm



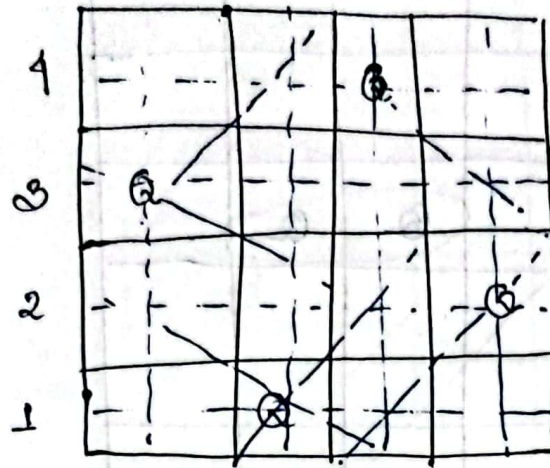
- This the reason why genetic Algo has been introduced. As here the traversal from one state to another doesn't need to be its adjacent state / child node.
- It traverse, completely to different state from one state
 - Work with only fit state.

Let create a vaccine for a disease using a immune gene.



n-queen problem

When $n=4$, So board = $n^2 = 4^2 = 16$



3 1 4 2 — State

Let take $n=8$

∴ For this we will take 4-state board so we will get 4

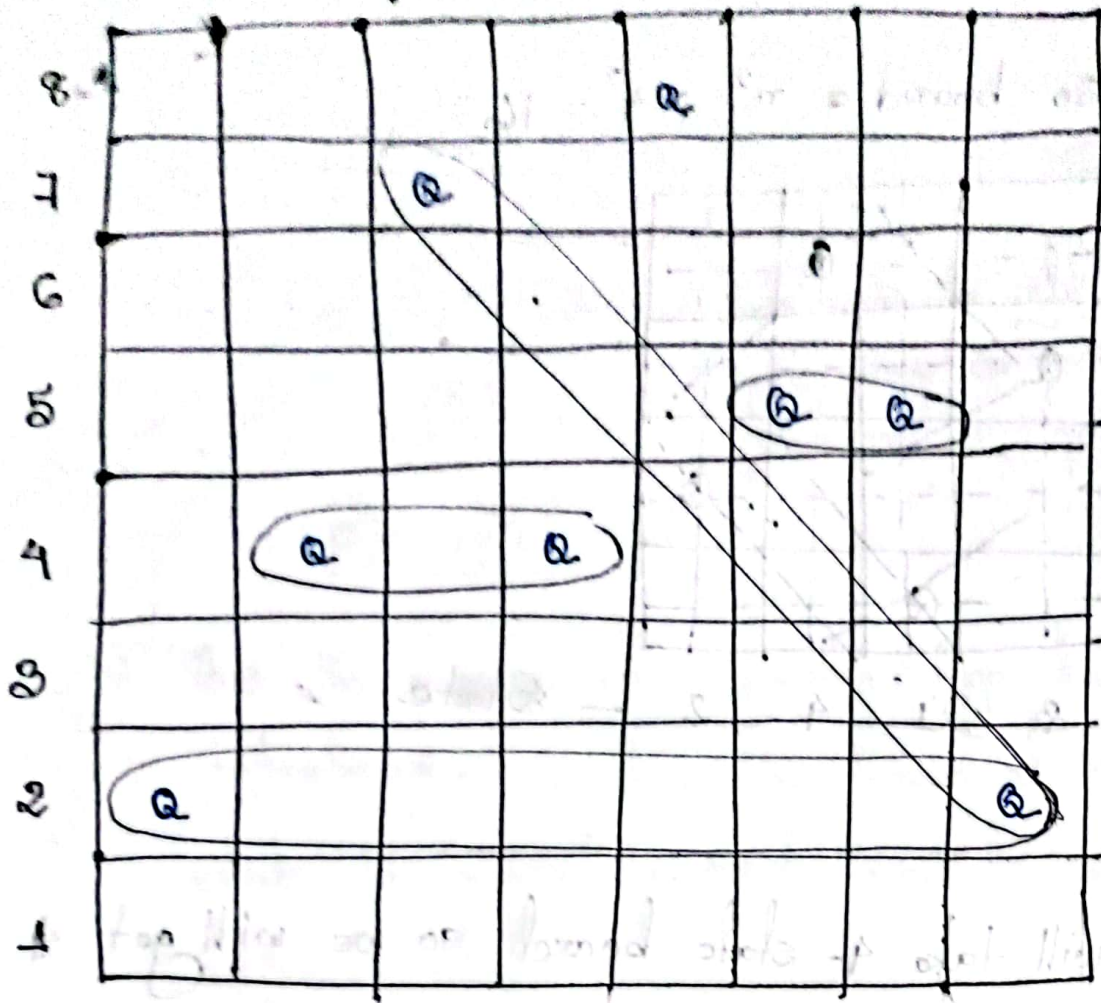
State :

Gene
 ← ② 4 7 4 8 5 5 2 → Chromosome 1 (each element of chrom-
 3 2 7 5 2 4 1 1 → " 2 osome/sample is gene)
 2 4 4 1 5 1 2 4 → 3
 8 2 5 4 2 2 1 2 → 4

Fitness Function = no of non-clashing pairs

↳ whose non clashing pairs is more,
 that board is more fit

Board State of chromosome 1 :



So clashing pair is = 4

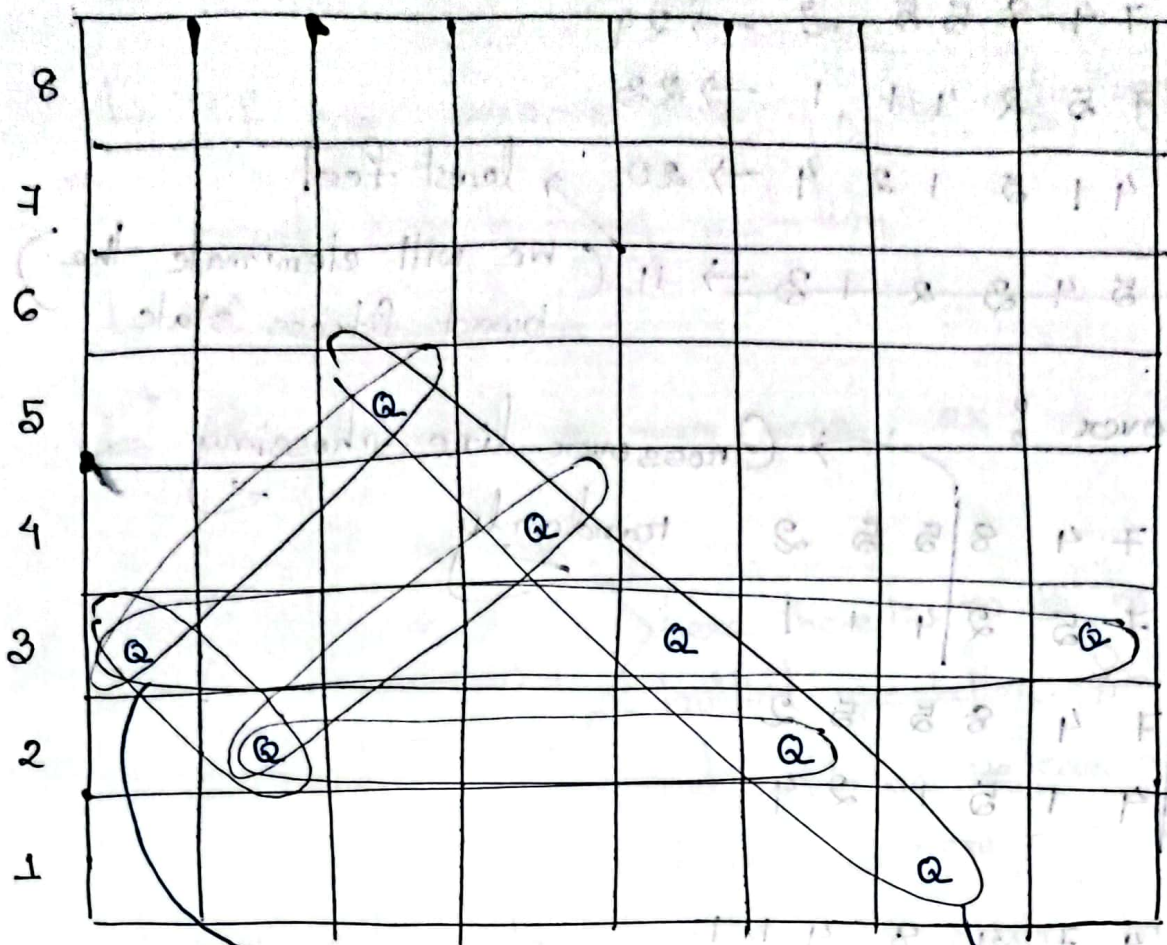
Total Pair = ${}^8C_2 = 28$

Non clashing pair = $28 - 4 = 24$

$\rightarrow {}^2C_2 + {}^2C_2 + {}^2C_2 + {}^2C_2 \rightarrow \left\{ \begin{array}{l} {}^2C_2 \text{ because paired} \\ \text{between 2} \end{array} \right\}$

$= 1 + 1 + 1 + 1$

Board State of Chromosome, 4.0



$$\text{No of clashing pair} = 1 + {}^3C_2 + 1 + 1 + 1 + {}^5C_2$$

$$= 1 + 3 + 3 + 10 = 17$$

$$\therefore \text{No of non clashing pair} = 28 - 17 = 11$$

$$\rightarrow \text{So for } n\text{-Queen making pair, total pair} = \boxed{{}^nC_2}$$

Step 1

So the fitness of the states are :

2 4 7 4 8 5 5 2 → 24

8 2 7 5 2 4 1 1 → 28

2 4 4 1 5 1 2 4 → 20

least fit

~~3 2 5 4 3 2 1 3 → 11~~

(We will eliminate the worst fitness state)

Step 2

Cross over :

Crossover line choosing randomly

2 4 7 4 8 | 5 5 2

8 2 7 5 2 | 4 1 1

2 4 | 7 4 8 5 5 2

2 4 | 4 1 5 1 2 4

2 4 7 4 8 4 1 1

8 2 7 5 2 5 5 2

2 4 4 1 5 1 2 4

2 4 7 4 8 5 5 2

Step 3

Mutation :

2 4 7 4 (8) 4 1 1 → 2 4 7 4 5 4 1 1

8 (2) 7 5 2 5 5 2 → 8 3 7 5 2 5 5 2

2 4 4 1 5 1 2 4 → 2 4 4 1 5 1 2 4

2 4 7 4 8 5 5 (2) → 2 4 7 4 8 5 5 2

Why Mutation ?

- ↳ To diversified the state
- ↳ If any gene is missing, the only way to insert that is doing mutation
- ↳ To remove clash
- ↳ Can introduce a new gene, ex-

1 4 4 2
1 2 2 1
2 4 2 4
4 1 1 4

here there is no 3 in the initial population. So without mutation we can never induce 3 here.

This 3 step will repeat again and again in a loop, unless we get the desired result.

initial state
eval time for this
state is 10

11	10	7	3	0	0	0	0	0	0
0	0	0	0	1	1	1	1	1	0
0	0	0	0	1	1	0	0	0	0
0	0	0	0	0	0	1	1	1	0
1	1	1	1	0	0	0	0	1	0

fitness = 10

(Lecture-7)

0/1 Knapsack Problem

→ ইয় নিচ else বাদ দিবে।

Steps:

Selection

Crossover

Mutation

Selection of Chromosome for knapsack:

ABCD
CD
AB
AEFGH

As there is chromosome of different length, the crossover operation will be problematic.

→ So to solve the problem, we will encode chromosome with binary number.

	A	B	C	D	E	F	G	H
c ₁	1	1	1	1	0	0	0	0
c ₂	0	0	1	1	0	0	0	0
c ₃	1	1	0	0	0	0	0	0
c ₄	1	0	0	0	1	1	1	1

fitness function
will not work here
as n-Queen

∴ Here fitness = Total reward

if $C_1(w) > \text{max-weight}$
 $f(C_1) = 0$

else
 $f(C_1) = C_1(R)$

Fitness :

$$C_1 = f(C_1) = 75 > 12 \therefore f(C_1) = 0 \text{ As } C_1(w) > 12$$

$$C_2 = f(C_2) = 50 > 12 \therefore f(C_2) = 0 \text{ " } C_2(w) > 12$$

$$C_3 = f(C_3) = 25 > 12 \therefore f(C_3) = 25 \text{ " } C_3(w) < 12$$

$$C_4 = f(C_4) = 25 > 12 \therefore f(C_4) = 0 \text{ " } C_4(w) > 12$$

incase all become 0 we will take the lowest one.

As tie occurs will use different approach.

$$f(C_1) = 0 - |\text{max-weight} - C_1(w)|$$

$$= |12 - 14| = 2$$

$$f(C_2) = |12 - 19| = 7$$

$$f(C_3) = |12 - 3| = 25$$

$$f(C_4) = |12 - 19| = 7$$

After that do crossover & mutation

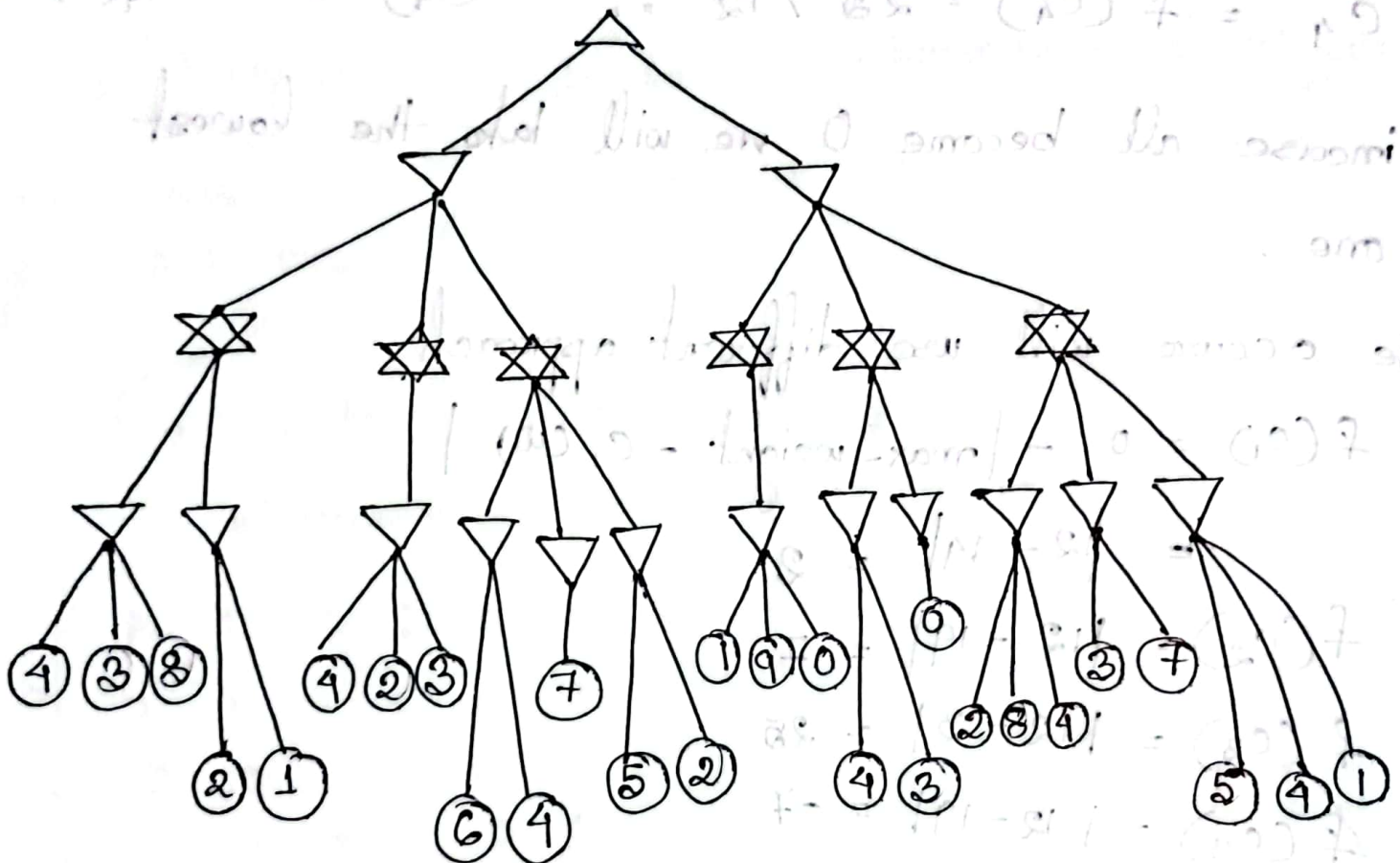
Problem :

$$(x_2 + x_4) - (x_1 + x_3) = 50$$

$$x_1, x_2, x_3, x_4$$

$$\therefore F(c_1) = |(x_2 + x_4) - (x_1 + x_3)| - 50$$

Alpha-Beta Pruning



Quiz 2 → Games

Quiz 3 → Genetic Algorithm (Next Thursday)